ENOSTAVNI UREJEVALNI alg.

- °SELECTION SORT → po vrsti izbiramo i-ti najmanjši element.
- °INSERTION SORT → po velikosti tabele i={1…n} vstavljamo elemente.
- °BUBBLE SORT → urejamo sosedne pare dokler ni sortiran.

časovne zahtevnosti $O(n^2)$.

NAPREDNI UREJEVALNI alg.

- °MERGESORT → razdeli seznam na polovico do n=1, nato po dva "merga".
- °QUICKSORT → razdeli seznam na majhne in velike glede na pivot do n=1… (MERGESORT)
- °HEAPSORT → selectionsort s hranjenjem elementov, da čas dostopa zmanjšamo na $O(\log n)$.
- °COUNTING SORT → za majhne sezname. č.z. $O(n)$, p.z. $O(\text{max el.})$
- °RADIX SORT → predvsem za strings, tu sortira glede na črko.

ostali večino $O(n \log n)$

binary search → najhitrejše isk.

HEAP - DVOJIŠKA KOPICA

ABSTR. POD. TIPI

AMORTIZIRANA ZAHT.

↳

- □SET (množica)
- □SLOVAR → {key, pair}
- □ZGOŠČENA TABELA
  - ↳ zgoščevalna tabela

DREVESA

OBHODI: premi, vmesni, obratni, nivojski

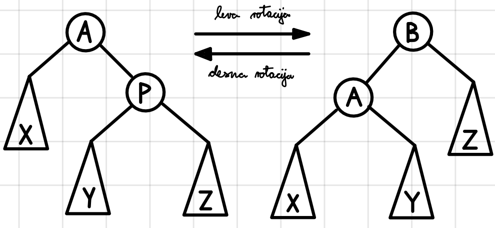
VRSTE: dvojiška (k-tiška), polno, iskalno urejena/neurejena, črkovna/znakovna (trie).

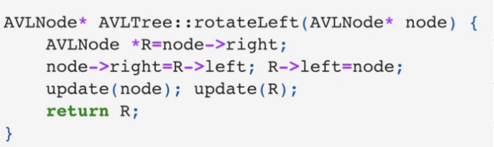
BLOKI: število blokov $\lceil n/B \rceil$

predobdelava $O(n), O(n)$

poizvedba $O(n/B+B), O(1) \rightarrow \overset{\text{optimalen}}{B} = \sqrt{n}$

AVL drevesa:





```
AVLNode* AVLTree::rotateLeft(AVLNode* node) {
    AVLNode *R=node->right;
    node->right=R->left; R->left=node;
    update(node); update(R);
    return R;
}
```

RDEČE-ČRNA: prazna črna, rdeče nima otrok, novo vozlišče je rdeče.

2-3 drevo: dva ali trije otroki

B-drevo: posplošena 2-3 drevesa - red m pomeni največ m otrok. (uporaba: pod. baze)

NAKLJUČNO - "treap", LOMLJENO - "splay"

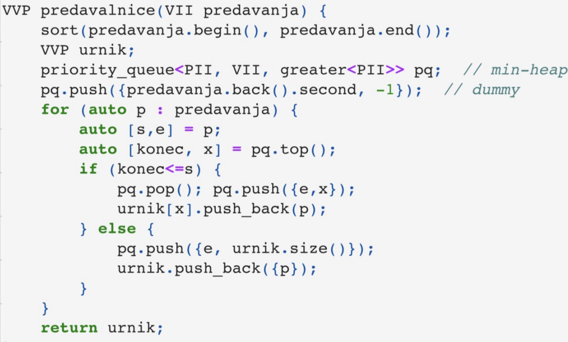
POŽREŠNI alg.

„v vsakem koraku se odločimo za izbiro, ki zgleda najbolj obetavno“

ČRPALKE → če je mogoče doseči naslednjo, preskočimo $O(n)$?

IZBIRA AKT. → izberemo najzgodnejši konec. $O(n \log n)$

REZERV. UČ. →



```
VVP predavalnice(VII predavanja) {
    sort(predavanja.begin(), predavanja.end());
    VVP urnik;
    priority_queue<PII, VII, greater<PII>> pq;  // min-heap
    pq.push({predavanja.back().second, -1});  // dummy
    for (auto p : predavanja) {
        auto [s,e] = p;
        auto [konec, x] = pq.top();
        if (konec<=s) {
            pq.pop(); pq.push({e,x});
            urnik[x].push_back(p);
        } else {
            pq.push({e, urnik.size()});
            urnik.push_back({p});
        }
    }
    return urnik;
}
```

DATOTEKE na traku: utež $\frac{\text{velikost}}{\text{pog. dost.}}$

(dokažemo s primeri 2 dat. in s=1 in f=1)

MIN. ZAMUDE: najzgodnejši rok

DOKAZOVANJE PRAVILNOSTI

↳ PREDNOST (stay ahead) po vsakem koraku rešitev tako dobra kot opt.

↳ ZAMENJAVA (exchange arg.)

1. predpostavimo da obstaja boljša rešitev > pož.
2. arg. da bi lahko izbrali pož. rešitev in rešitev nebi bila nič slabša.
3. našli smo protislovje ⬆ 4. □ dokazali

↳ STRUKTURA (str. arg.)

dokažemo neko strukturno lastnost optimalne reš., ki predstavlja mejo in dokažemo, da jo požrešna reš doseže.

GRAFI (usm., enost., cik./acik., gosti./red.)

(drevesa, polni grafi, reg. grafi, dvodelni grafi)

(sprehod, obhod, steza, pot, cikel) $n=|V|$ št. pov.

(seznam pov., sez. sosedov, matrika sos.) $e=|E|$ št. voz.

PREISKOVANJE

- °BFS: gre nivojsko - sosedi, sosedi sosedov…
- °DFS: gre prvo do konca pri enem sosedu.

TOPOLOŠKO UR. KRITIČNA POT

$O(n+e)$ enak alg. obrnjen

UTEŽENI GRAFI

DIJKSTRA prost. $O(n)$, čas. $O(n^2)$ nav., $\overset{\text{s heap}}{O(e \log n)}$

↳ iskanje najkrajše poti. z Bucket Q $O(e+nc)$

negativne uteži imajo načeloma smisel le v usmerjenih grafih.

VPETA DREVESA

join $O(1)$, find $O(\log n)$

DISJUNKTNE MN.

MIN. VPETO DREVO „najmanjša prerezna pov. vedno del min. vpetega drevesa.“

- °PRIM (pož. alg.) → širjenje iz izh. vozlišča
- °KRUSKAL (-||-) → dodajanje vozlišč ◢ → povezava ne sme ustvariti cikla!

č.z. $O(mn)$, $O(m \log n)$ s pod. strukturo disjunktnih množic

DELI in VLADAJ

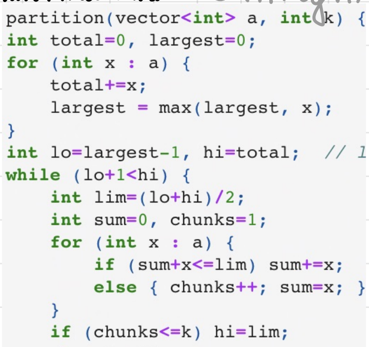
problem razdelimo na več manjših podproblemov in rezultate manjših zdr. v reš. večjega.

KROVNI IZREK → običajno razbijemo probleme n na n/b. $T(n)=aT(n/b)+f(n)$ $(a \geq 1, b > 1)$

a - št. podpr., b - vel. podpr. ?…

°ENAKOMERNO RAZBITJE SEZ.

min. ↷ max. vsota $O(n \log n)$



```
partition(vector<int> a, int k) {
int total=0, largest=0;
for (int x : a) {
    total+=x;
    largest = max(largest, x);
}
int lo=largest-1, hi=total;  // 1
while (lo+1<hi) {
    int lim=(lo+hi)/2;
    int sum=0, chunks=1;
    for (int x : a) {
        if (sum+x<=lim) sum+=x;
        else { chunks++; sum=x; }
    }
    if (chunks<=k) hi=lim;
    else lo=lim;
```

°K-TI ELEMENT $O(n)$ v C++ nth_element(a.begin(),…)

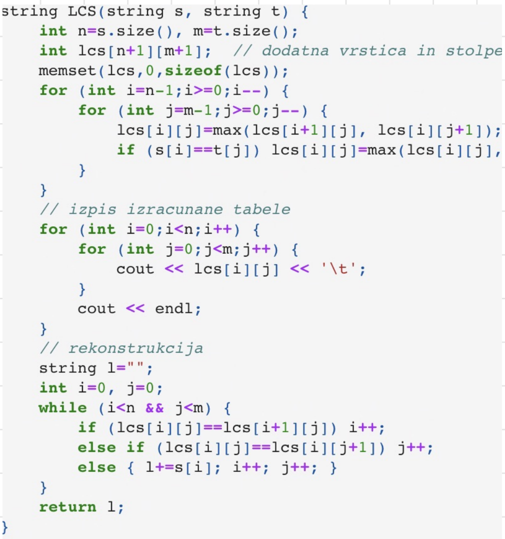
DINAMIČNO prog.

lastnosti:
- neodvisnost podproblemov
- optimalna podstruktura
- ponavljanje podproblemov

ne računaj enakih stvari večkrat!

fib., žablji skoki, rezanje palice, pot v mreži…

NAJDALJŠE SKUPNO ZAP.



```
string LCS(string s, string t) {
    int n=s.size(), m=t.size();
    int lcs[n+1][m+1];  // dodatna vrstica in stolpe
    memset(lcs,0,sizeof(lcs));
    for (int i=n-1;i>=0;i--) {
        for (int j=m-1;j>=0;j--) {
            lcs[i][j]=max(lcs[i+1][j], lcs[i][j+1]);
            if (s[i]==t[j]) lcs[i][j]=max(lcs[i][j],
        }
    }
    // izpis izracunane tabele
    for (int i=0;i<n;i++) {
        for (int j=0;j<m;j++) {
            cout << lcs[i][j] << '\t';
        }
        cout << endl;
    }
    // rekonstrukcija
    string l="";
    int i=0, j=0;
    while (i<n && j<m) {
        if (lcs[i][j]==lcs[i+1][j]) i++;
        else if (lcs[i][j]==lcs[i][j+1]) j++;
        else { l+=s[i]; i++; j++; }
    }
    return l;
}
```

NASLEDNJA STRAN

obrni!

nahrbtnik še iz te snovi!

RAČUNSKA GEOMETRIJA

PRESEČIŠČA: točka S in pr. P $P_0S \times V_p = 0$

premici P in R: $P_0 + aV_p = R_0 + bV_R$ $\boxed{proj_b a = \frac{a \cdot b}{\|b\|^2} b}$

POVRŠINA VEČKOTNIKA:

osnova: $p = 1/2 |AB \times AC|$

pri nekonveksnem koordinate razdelimo po naraščajočih x in na vsakem intervalu računamo ploščino → $\sum$

VSEBOVANOST TOČKE:

ciljamo skozi točko, če seka sodo število premic je točka vsebovana.

KONVEKSNA OVOJNICA:

ravijanje darila

grahamov pregled

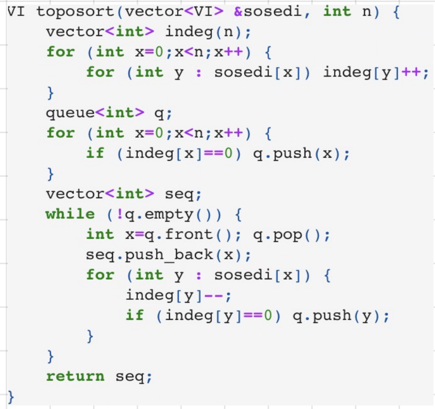
pri obeh je uporabljen vektorski produkt … če je $AC \times AB > 0$ je kot levo, drugače je desno cross(P1| P

↳ za sortiranje kotov uporabimo atan2()

indentifikacija stranic

"se vse točke nahajajo na isti strani daljice?"

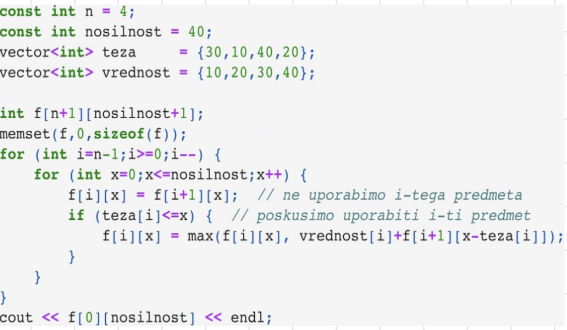
$O(\{n^3, n^2, n\log n\})$



```
VI toposort(vector<VI> &sosedi, int n) {
    vector<int> indeg(n);
    for (int x=0;x<n;x++) {
        for (int y : sosedi[x]) indeg[y]++;
    }
    queue<int> q;
    for (int x=0;x<n;x++) {
        if (indeg[x]==0) q.push(x);
    }
    vector<int> seq;
    while (!q.empty()) {
        int x=q.front(); q.pop();
        seq.push_back(x);
        for (int y : sosedi[x]) {
            indeg[y]--;
            if (indeg[y]==0) q.push(y);
        }
    }
    return seq;
}
```

↑ topološko urejanje od prej…

NAHRBTNIK



```
const int n = 4;
const int nosilnost = 40;
vector<int> teza     = {30,10,40,20};
vector<int> vrednost = {10,20,30,40};

int f[n+1][nosilnost+1];
memset(f,0,sizeof(f));
for (int i=n-1;i>=0;i--) {
    for (int x=0;x<=nosilnost;x++) {
        f[i][x] = f[i+1][x];  // ne uporabimo i-tega predmeta
        if (teza[i]<=x) {  // poskusimo uporabiti i-ti predmet
            f[i][x] = max(f[i][x], vrednost[i]+f[i+1][x-teza[i]]);
        }
    }
}
cout << f[0][nosilnost] << endl;
```

C++ Sheet CHEAT

STDIN/OUT cout << " "; cin >> a;

STRING string str; str.size(); //velikost

PAIR pair<int,int> p

AUTO (določitev tipa) auto a; obnaša se kot tip.

ITERATOR for(vector<int>::iterator it = v.begin(); it != v.end();) *it

RAZPAKIRANJE for(auto [x,y] : pairs) cout << x << y

VRSTA queue<T>

front(); back(); push(n); pop(); empty();…

SKLAD stack<T>

top(); push(n); pop(); empty(); …

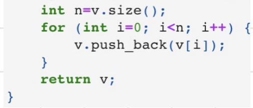
MNOZICA map<string, int>

begin(); [illegible] (); insert(…); erase(…); find(…); /*w_key*/ count(…);…

POV. SEZNAM list<T>

front(); back(); empty(); push_front(); push_back(); insert(…);… (sort reverse)

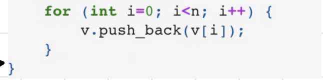
REFERENCE int x = 10; int &y = x; // x je enak y



```
auto podvoji(vector<int> v) {
    int n=v.size();
    for (int i=0; i<n; i++) {
        v.push_back(v[i]);
    }
    return v;
}
```

← ta funkcija ne spremeni vrednosti prvotnega seznama!

ta pa jo →



```
void podvoji_ref(vector<int> &v) {
    int n=v.size();
    for (int i=0; i<n; i++) {
        v.push_back(v[i]);
    }
}
```

f(int& poz) poz=0; // u get the idea …

MIN,MAX min(5,2); max({5,2,0});

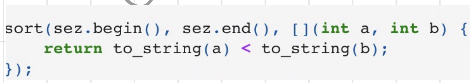
*min_element(v.begin(), v.end());

SORT

sort(v.begin(), v.end());

#include <algorithm>

ANONIMNE funkcije



```
sort(sez.begin(), sez.end(), [](int a, int b) {
    return to_string(a) < to_string(b);
});
```

SEZNAM vector<T>

resize(n); capacity(); push_back(n); pop_back(); clear(); swap(V);…

za predmet Algoritmi in podatkovne strukture 1, FRI UNI ULJ

: